CO4-AT1-Industrial Problem Solving

Hospital Appointment Management System

1. GUI Components

Component	Purpose
JFrame	Main application window
JLabel	Displays field names
JTextField	Accepts patient information
JComboBox	Selects doctor or appointment type
JButton	Performs Book, Update, and Cancel operations
JTable	Displays appointment records
JScrollPane	Provides scrolling for the table
JMenuBar	Provides application menus
JOptionPane	Displays confirmation and error messages

2. Event Handling

Event listeners are used to make the application interactive.

For example, when the **Book Appointment** button is clicked:

1. Patient information is read from the text fields.
2. The entered data is validated.
3. A new appointment record is created.
4. The record is added to the table.
5. A confirmation message is displayed.
6. Input fields can be cleared for the next patient.

Similarly, the **Update** button modifies the selected appointment, while the **Cancel** button removes it after obtaining confirmation from the user.

3. Dynamic Patient Data

A DefaultTableModel can be used with JTable to manage patient appointment records dynamically.

```
DefaultTableModel model = new DefaultTableModel(
   new String[]{"Patient ID", "Name", "Doctor", "Date"}, 0);
JTable table = new JTable(model);
```

New records can be added using:

```
model.addRow(new Object[]{
   "P101", "Anitha", "Dr. Kumar", "17-08-2026"
});
```

Records can also be updated or deleted without recreating the entire GUI.

4. Scalability Analysis

When the number of patients increases, storing all data directly inside GUI components becomes inefficient. A scalable system should separate the user interface from the data-management layer.

A suitable architecture is:

User

↓

Swing GUI

↓

Event Listener / Controller

↓

Appointment Service

↓

Database

For a small demonstration application, ArrayList or DefaultTableModel is sufficient. For a real hospital system, a database such as MySQL or PostgreSQL should be used.

Database storage provides:

- Large-scale patient storage.
- Faster searching and filtering.
- Persistent appointment records.
- Prevention of data loss when the application closes.
- Multiple-user support.
- Better backup and recovery.

Indexes can be created for fields such as Patient ID, Doctor ID, and Appointment Date to improve searching when the number of records becomes large.

CODE:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
public class Main extends JFrame {
   JTextField patientIdField;
   JTextField nameField;
   JTextField ageField;
   JTextField phoneField;
   JTextField dateField;
   JTextField timeField;
   JTextField searchField;
   JComboBox<String> doctorBox;
   JTable table;
   DefaultTableModel model;
   JButton bookButton;
   JButton updateButton;
   JButton cancelButton;
   JButton clearButton;
   JButton searchButton;
   public Main() {
      setTitle("Hospital Appointment Management System");
      setSize(950, 600);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      setLocationRelativeTo(null);
      JMenuBar menuBar = new JMenuBar();
      JMenu fileMenu = new JMenu("File");
      JMenuItem clearMenuItem = new JMenuItem("Clear");
      JMenuItem exitMenuItem = new JMenuItem("Exit");
      clearMenuItem.addActionListener(e -> clearFields());
      exitMenuItem.addActionListener(e -> System.exit(0));
```

```
fileMenu.add(clearMenuItem);
fileMenu.addSeparator();
fileMenu.add(exitMenuItem);
JMenu helpMenu = new JMenu("Help");
JMenuItem aboutItem = new JMenuItem("About");
aboutItem.addActionListener(e ->
    JOptionPane.showMessageDialog(
        this,
        "Hospital Appointment Management System\n" +
        "Java Swing Application"
    )
);
helpMenu.add(aboutItem);
menuBar.add(fileMenu);
menuBar.add(helpMenu);
setJMenuBar(menuBar);
JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
JLabel titleLabel = new JLabel(
    "HOSPITAL APPOINTMENT MANAGEMENT SYSTEM",
    JLabel.CENTER
);
titleLabel.setFont(
    new Font("Arial", Font.BOLD, 20)
);
mainPanel.add(
    titleLabel,
    BorderLayout.NORTH
);
JPanel detailsPanel =
    new JPanel(new GridLayout(7, 2, 8, 8));
detailsPanel.setBorder(
```

```
        BorderFactory.createTitledBorder(
            "Appointment Details"
        )
    );
    detailsPanel.add(
        new JLabel("Patient ID:")
    );
    patientIdField = new JTextField();
    detailsPanel.add(patientIdField);
    detailsPanel.add(
        new JLabel("Patient Name:")
    );
    nameField = new JTextField();
    detailsPanel.add(nameField);
    detailsPanel.add(
        new JLabel("Age:")
    );
    ageField = new JTextField();
    detailsPanel.add(ageField);
    detailsPanel.add(
        new JLabel("Phone:")
    );
    phoneField = new JTextField();
    detailsPanel.add(phoneField);
    detailsPanel.add(
        new JLabel("Doctor:")
    );
    doctorBox = new JComboBox<>(
        new String[]{
            "Dr. Kumar - General Physician",
            "Dr. Priya - Cardiologist",
```

```
            "Dr. Ravi - Neurologist",
            "Dr. Anitha - Pediatrician"
        }
);
detailsPanel.add(doctorBox);
detailsPanel.add(
        new JLabel("Appointment Date:")
);
dateField = new JTextField();
detailsPanel.add(dateField);
detailsPanel.add(
        new JLabel("Appointment Time:")
);
timeField = new JTextField();
detailsPanel.add(timeField);
JPanel leftPanel =
        new JPanel(new BorderLayout());
leftPanel.add(
        detailsPanel,
        BorderLayout.NORTH
);
JPanel buttonPanel = new JPanel(new FlowLayout());
bookButton = new JButton("Book Appointment");
updateButton =
        new JButton("Update Appointment");
cancelButton = new JButton("Cancel Appointment");
clearButton = new JButton("Clear");
buttonPanel.add(bookButton);
buttonPanel.add(updateButton);
buttonPanel.add(cancelButton);
buttonPanel.add(clearButton);
```

```
leftPanel.add(
    buttonPanel,
    BorderLayout.CENTER
);
mainPanel.add(
    leftPanel,
    BorderLayout.WEST
);
String[] columns = {
    "Patient ID",
    "Name",
    "Age",
    "Phone",
    "Doctor",
    "Date",
    "Time"
};
model = new DefaultTableModel(
   columns,
    0
);
table = new JTable(model);
table.setSelectionMode(
    ListSelectionModel.SINGLE_SELECTION
);
JScrollPane scrollPane =
    new JScrollPane(table);
scrollPane.setBorder(
    BorderFactory.createTitledBorder(
        "Appointment Records"
    )
```

```
);
mainPanel.add(
    scrollPane,
    BorderLayout.CENTER
);
JPanel searchPanel =
    new JPanel(new FlowLayout());
searchPanel.add(
    new JLabel("Search Patient ID:")
);
searchField = new JTextField(15);
searchPanel.add(searchField);
searchButton =  new JButton("Search");
searchPanel.add(searchButton);
JButton showAllButton =
    new JButton("Show All");
searchPanel.add(showAllButton);
mainPanel.add(
    searchPanel,
    BorderLayout.SOUTH
);
bookButton.addActionListener(
    e -> bookAppointment()
);
updateButton.addActionListener(
    e -> updateAppointment()
);
cancelButton.addActionListener(
    e -> cancelAppointment()
);
clearButton.addActionListener(
```

```
        e -> clearFields()
);
searchButton.addActionListener(
        e -> searchAppointment()
);
showAllButton.addActionListener(
        e -> showAllAppointments()
);
table.getSelectionModel()
        .addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) {
                int row =
                    table.getSelectedRow();
                if (row != -1) {
                    patientIdField.setText(
                        model.getValueAt(
                            row, 0
                        ).toString());
                    nameField.setText(
                        model.getValueAt(
                            row, 1
                        ).toString()
                    );
                    ageField.setText(
                        model.getValueAt(
                            row, 2
                        ).toString()
                    );
                    phoneField.setText(
                        model.getValueAt(
                            row, 3
```

```
                    ).toString()
                );
                doctorBox.setSelectedItem(
                    model.getValueAt(
                        row, 4
                    ).toString()
                );
                dateField.setText(
                    model.getValueAt(
                        row, 5
                    ).toString()
                );
                timeField.setText(
                    model.getValueAt(
                        row, 6
                    ).toString()
                );
            }
        }
    });
  add(mainPanel);
  setVisible(true);
}
private void bookAppointment() {
  try {
    String patientId = patientIdField.getText().trim();
    String name = nameField.getText().trim();
    String ageText = ageField.getText().trim();
    String phone = phoneField.getText().trim();
    String doctor = doctorBox.getSelectedItem().toString();
    String date = dateField.getText().trim();
```

```
String time = timeField.getText().trim();
if (patientId.isEmpty()) {
   throw new IllegalArgumentException(
        "Please enter Patient ID."
   );
}
if (name.isEmpty()) {
   throw new IllegalArgumentException(
        "Please enter patient name."
   );
}
if (ageText.isEmpty()) {
   throw new IllegalArgumentException(
        "Please enter age."
   );
}
int age;
try {
   age = Integer.parseInt(ageText);
} catch (NumberFormatException e) {
   throw new IllegalArgumentException(
        "Age must be a number."
   );
}
if (age <= 0 || age > 120) {
   throw new IllegalArgumentException(
        "Please enter a valid age."
   );
}
if (!phone.matches("\\d{10}")) {
   throw new IllegalArgumentException(
```

```
            "Phone number must contain 10 digits."
        );
    }
    if (date.isEmpty()) {
        throw new IllegalArgumentException(
            "Please enter appointment date."
        );
    }
    if (time.isEmpty()) {
        throw new IllegalArgumentException(
            "Please enter appointment time."
        );
    }
    for (int i = 0; i < model.getRowCount(); i++) {
        String existingId =
            model.getValueAt(i, 0).toString();
        if (existingId.equalsIgnoreCase(patientId)) {
            throw new IllegalArgumentException(
                "Patient ID already exists."
            );
        }
    }
    model.addRow(
        new Object[]{
            patientId,
            name,
            age,
            phone,
            doctor,
            date,
            time
```

```
            }
        );
        JOptionPane.showMessageDialog(
            this,
            "Appointment booked successfully!",
            "Success",
            JOptionPane.INFORMATION_MESSAGE
        );
        clearFields();
    } catch (IllegalArgumentException e) {
        JOptionPane.showMessageDialog(
            this,
            e.getMessage(),
            "Input Error",
            JOptionPane.ERROR_MESSAGE
        );
    }
}
private void updateAppointment() {
    int row = table.getSelectedRow();
    if (row == -1) {
        JOptionPane.showMessageDialog(
            this,
            "Please select an appointment from the table."
        );
        return;
    }
    try {
        String patientId = patientIdField.getText().trim();
        String name = nameField.getText().trim();
        String ageText = ageField.getText().trim();
```

```
String phone = phoneField.getText().trim();
String doctor = doctorBox.getSelectedItem().toString();
String date = dateField.getText().trim();
String time = timeField.getText().trim();
if (patientId.isEmpty()) {
   throw new IllegalArgumentException(
        "Patient ID cannot be empty."
   );
}
if (name.isEmpty()) {
   throw new IllegalArgumentException(
        "Patient name cannot be empty."
   );
}
int age;
try {
   age = Integer.parseInt(ageText);
} catch (NumberFormatException e) {
   throw new IllegalArgumentException(
        "Age must be a number."
   );
}
if (age <= 0 || age > 120) {
   throw new IllegalArgumentException(
        "Invalid age."
   );
}
if (!phone.matches("\\d{10}")) {
   throw new IllegalArgumentException(
        "Phone number must contain 10 digits."
   );
```

```
        }
        if (date.isEmpty()) {
           throw new IllegalArgumentException(
                "Please enter appointment date.");
        }
        if (time.isEmpty()) {
           throw new IllegalArgumentException(
                "Please enter appointment time.");
        }
        int choice =
             JOptionPane.showConfirmDialog(
                  this,
                  "Update this appointment?",
                  "Update Confirmation",
                  JOptionPane.YES_NO_OPTION  );
        if (choice ==  JOptionPane.YES_OPTION) {model.setValueAt(patientId, row,0);
           model.setValueAt(name,row,1);
           model.setValueAt(age, row, 2);
           model.setValueAt(phone, row, 3);
           model.setValueAt(doctor, row, 4 );
           model.setValueAt(date, row, 5);
           model.setValueAt(time, row, 6);
           JOptionPane.showMessageDialog(this, "Appointment updated successfully!");
           clearFields();
        }
      } catch (IllegalArgumentException e) {
         JOptionPane.showMessageDialog(this, e.getMessage(),"Input
Error",JOptionPane.ERROR_MESSAGE);
      }
   }
   private void cancelAppointment() {
```

```
    int row = table.getSelectedRow();
    if (row == -1) {
      JOptionPane.showMessageDialog(
          this,
          "Please select an appointment." );
      return;
    }
    String patientId =model.getValueAt(row, 0).toString();
    int choice = JOptionPane.showConfirmDialog(this, "Cancel appointment for Patient ID " +
patientId + "?",  "Cancel Appointment",JOptionPane.YES_NO_OPTION);
    if (choice == JOptionPane.YES_OPTION) {
      model.removeRow(row);
      JOptionPane.showMessageDialog(this, "Appointment cancelled successfully.");
      clearFields();
    }
  }
  private void searchAppointment() {
    String searchId = searchField.getText().trim();
    if (searchId.isEmpty()) {
      JOptionPane.showMessageDialog(
          this,
          "Please enter Patient ID to search.");
      return;
    }
    boolean found = false;
    for (int i = 0;
       i < model.getRowCount();
       i++) {
      String patientId = model.getValueAt( i, 0).toString();
      if (patientId.equalsIgnoreCase(searchId)) {
        table.setRowSelectionInterval(i,i);
```

```
            found = true;
            break;
        }
    }
    if (!found) {
        JOptionPane.showMessageDialog(
            this,
            "Patient appointment not found."  );
    }
}
private void showAllAppointments() {
    table.clearSelection();
    searchField.setText("");
    JOptionPane.showMessageDialog(
            this,
            "Showing all appointment records.");
}
private void clearFields() {
    patientIdField.setText("");
    nameField.setText("");
    ageField.setText("");
    phoneField.setText("");
    dateField.setText("");
    timeField.setText("");
    doctorBox.setSelectedIndex(0);
    table.clearSelection();
}
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new Main();});
}}
```